공유 가상 메모리 시스템의 메모리 일관성(Coherence)

Kai Li, Paul Hudak

ACM Transactions on Computer Systems, 1989

윤태영 문순원

M1522.006300 분산 시스템

2026년 10월 19일

발표 순서

1부 | 윤태영

1. 서론
2. 공유 가상 메모리
3. 중앙집중형 알고리즘

2부 | 문순원

1. 분산 알고리즘
2. 성능 평가
3. 결론

1. 공유 가상 메모리의 목표

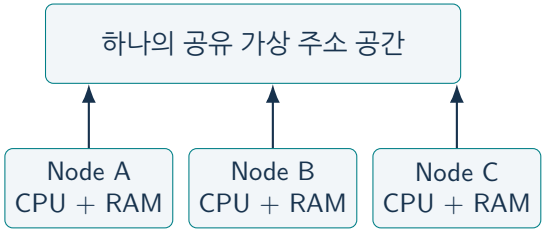
Message passing

프로그래머가 데이터 요청과 전달을 작성

Shared virtual memory

평범한 메모리 접근으로 공유 데이터 사용

```
x = shared_array[i]
```

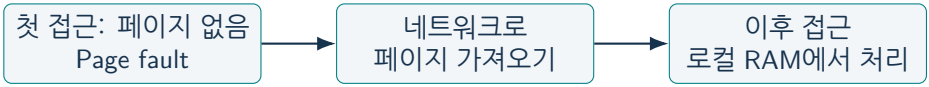


주소 공간은 공유하지만, RAM은 분리되어 있다.

공유 데이터에 접근할 때 필요한 페이지 이동은 시스템이 처리한다.

출처: Li & Hudak (1989), Sections 1–2, pp. 321–325.

1. 페이지 이동과 지역성



- 공유 주소 공간을 **페이지** 단위로 나눈다.
- 페이지는 필요에 따라 노드 사이에서 이동하거나 복제된다.
- 권한이 유지되는 동안에는 일반 로컬 메모리 접근을 사용한다.

한 번 가져온 데이터를 여러 번 쓰면 통신 비용을 줄일 수 있다.

출처: Li & Hudak (1989), Section 2, pp. 322–323; Section 3.1, pp. 325–326.

2. 페이지 복사본의 일관성

복사본을 조율하지 않으면

Node A	Node B
$x = 1$	$x = 0$

A가 수정한 뒤에도
B가 오래된 값을 읽음

페이지별 접근 규칙

여러 reader 또는 **하나의 writer**

쓰기 전에 다른 복사본을 무효화한다.

무효화: 복사본의 접근 권한을 제거.
오래된 바이트가 남아 있어도 사용할 수 없다.

쓰기 전에 다른 복사본을 무효화해 오래된 값의 사용을 막는다.

출처: Li & Hudak (1989), Section 2, p. 322; Sections 3–3.2, pp. 325–328.

2. MMU와 페이지 폴트 처리

각 노드의 페이지 권한

`nil`	접근 불가
`read`	읽기만 가능
`write`	읽기와 쓰기 가능

1. 권한이 없으면 **page fault**
2. **Fault handler**가 네트워크 요청
3. 원격 노드의 **server**가 요청 처리
4. 페이지와 권한 확보 후 원래 명령 재개

MMU는 자기 노드의 권한만 검사한다.

MMU가 권한 없는 접근을 감지하면 fault handler가 원격 페이지를 요청한다.

출처: Li & Hudak (1989), Section 3.1, p. 326; Section 3.3, pp. 328–329.

3. Centralized manager의 자료구조

중앙 Manager: `Info[p]`

owner: 페이지를 책임지는 노드

copy set: 읽기 복사본을 가진 노드들

lock: 같은 페이지의 요청 조율

각 노드: `PTable[p]`

access: 로컬 접근 권한

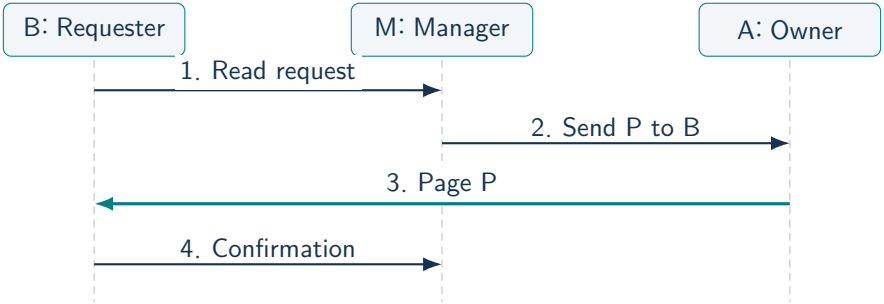
lock: 로컬 페이지 처리 조율

Manager는 한 노드가 맡는 역할.
실제 페이지 데이터는 owner가 전달한다.

Manager는 owner 위치를 기록하고, owner는 페이지 데이터를 전달한다.

출처: Li & Hudak (1989), Section 4.1, pp. 329–332; Figure 2.

3. 읽기 fault 처리 과정



Owner는 A로 유지. A와 B는 `read`. Copy set에 B 추가.

일반적인 비관리 노드의 읽기 fault: 4개 메시지.

출처: Li & Hudak (1989), Section 4.1, pp. 330–332; Figure 2.

3. 읽기 fault 의사코드

설명용 의사코드: B=requester, M=manager, A=현재 owner.
서로 다른 노드인 경로만 표시. `receive/wait`는 완료까지 대기.

B: Read fault handler

```
1 lock(Local[p])
2 send(M, READ(p, B))
3 receive(A, page p)
4 send(M, DONE(p))
5 access[p] = READ
6 unlock(Local[p])
7 retry faulting load
```

B의 lock은 페이지 수신과
권한 설정이 끝날 때까지 유지.

M: Read request 처리

```
1 lock(Info[p].lock)
2 Info[p].copies.add(B)
3 send(Info[p].owner, READ(p, B))
4 wait DONE(p) from B
5 unlock(Info[p].lock)
```

A: Owner의 read server

```
1 lock(Local[p])
2 access[p] = READ
3 send(B, page p)
4 unlock(Local[p])
```

복사본을 내보내기 전에 owner의 쓰기 권한을 제거한다.

출처: Li & Hudak (1989), Figure 2, pp. 330–331; nonmanager path, reorganized.

4. 쓰기 fault 처리 과정

1. B가 M에게 쓰기 요청
2. M이 읽기 복사본들을 **무효화**하고 완료 확인
3. M이 A에게 B로 페이지 전송 요청
4. A가 페이지를 전달하고 자기 접근 권한 포기
5. B가 수신 확인 후 쓰기 권한으로 실행 재개

완료 후 상태

`owner = B`

`copy set = ∅`

B: `write`
다른 노드: `nil`

읽기는 복사본 추가. 쓰기는 무효화와 ownership 이전.

출처: Li & Hudak (1989), Section 3.2, pp. 326–327; Section 4.1, pp. 330–332.

4. 쓰기 fault 의사코드

설명용 의사코드: B=requester, M=manager, A=현재 owner.
서로 다른 노드인 경로만 표시. `receive/wait`는 완료까지 대기.

B: Write fault handler

```
1 lock(Local[p])
2 send(M, WRITE(p, B))
3 receive(A, page p)
4 send(M, DONE(p))
5 access[p] = WRITE
6 unlock(Local[p])
7 retry faulting store
```

A는 local lock 아래 페이지를
전달하며 자기 접근 권한을 포기.

M: Write request 처리

```
1 lock(Info[p].lock)
2 invalidate(p, Info[p].copies)
3 wait all invalidation replies
4 Info[p].copies = {}
5 send(Info[p].owner, WRITE(p, B))
6 wait DONE(p) from B
7 Info[p].owner = B  // explicit update
8 unlock(Info[p].lock)
```

Owner 갱신은 본문의 설명에 따라 명시.
Figure 2에는 이 대입이 표시되어 있지 않음.

다른 복사본의 권한 제거와 이전 owner의 권한 포기 후 WRITE 허용.

출처: Li & Hudak (1989), Section 3.3, p. 329; Section 4.1 and Figure 2, pp. 330–332; adapted.

3–4. Manager lock과 로컬 lock

Manager의 `Info[p].lock`

여러 노드에서 들어온
같은 페이지의 요청을 조율

기본 방식에서는 완료 확인까지 유지

각 노드의 `PTable[p].lock`

그 노드의 **fault handler와 server**가
같은 페이지를 처리하는 순서 조율

응답 대기 중 들어오는 요청도 대기시킴

페이지 관리용 잠금이다. 일반적인 load/store마다 잡지 않는다.

출처: Li & Hudak (1989), Section 3.3, p. 328; Section 4.1, p. 331.

4. 읽기·쓰기 권한과 일관성

유지하려는 불변식: 각 페이지의 접근 권한

어떤 노드가 `WRITE`이면 다른 노드는 모두 `nil`.
유효한 복사본이 여러 개라면 모두 `READ`.

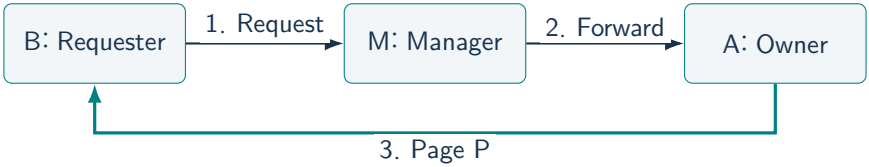
1. **읽기 추가:** owner도 `READ`로 바꾼 뒤 최신 페이지 복제
2. **쓰기 이전:** 읽기 복사본과 이전 owner의 권한 제거 후 `WRITE`
3. **요청 충돌:** 같은 페이지의 전달 과정은 lock과 완료 확인으로 조율

한 노드가 쓰기 권한을 가지면 다른 노드의 복사본은 모두 무효 상태다.

출처: Li & Hudak (1989), Sections 3.2–4.1, pp. 326–332; correctness intuition under the paper's assumptions.

5. Improved centralized algorithm

역할	기본 centralized	Improved centralized
Owner 위치 기록	Manager	Manager
Copy set 관리	Manager	Owner
페이지 전달 과정 조율	Manager 중심	Owner의 local lock
Manager에 완료 확인	필요	제거



일반적인 읽기 fault: 4개에서 3개 메시지로 감소.

출처: Li & Hudak (1989), Section 4.2, pp. 332–333; Appendix A, pp. 350–351.

5. 개선된 읽기 fault 의사코드

설명용 의사코드: B=requester, M=manager, A=현재 owner.
서로 다른 노드인 경로만 표시. `receive/wait`는 완료까지 대기.

B: Read fault handler

```
1 lock(Local[p])
2 send(M, READ(p, B))
3 receive(A, page p)
4 access[p] = READ
5 unlock(Local[p])
6 retry faulting load
```

Manager로 보내던
DONE 메시지가 사라짐.

M: Directory lookup

```
1 lock(ManagerLock)
2 forward READ(p, B) to Owner[p]
3 unlock(ManagerLock)
```

A: Owner의 read server

```
1 lock(Local[p])
2 copies[p].add(B)
3 access[p] = READ
4 send(B, page p)
5 unlock(Local[p])
```

Manager는 전달 완료를 기다리지 않고, owner가 복사본을 추적한다.

출처: Li & Hudak (1989), Section 4.2, pp. 332–333; Appendix A, p. 350; role-specific excerpt adapted.

5. 개선된 쓰기 fault 의사코드

설명용 의사코드: B=requester, M=manager, A=현재 owner.
서로 다른 노드인 경로만 표시. `receive/wait`는 완료까지 대기.

B: Write fault handler

```
1 lock(Local[p])
2 send(M, WRITE(p, B))
3 receive(A, page p, copies[p])
4 invalidate(p, copies[p])
5 wait all invalidation replies
6 access[p] = WRITE
7 copies[p] = {}
8 unlock(Local[p])
9 retry faulting store
```

B는 **페이지와 copy set을 함께** 받음.

M: Directory update

```
1 lock(ManagerLock)
2 forward WRITE(p, B) to Owner[p]
3 Owner[p] = B
4 unlock(ManagerLock)
```

A: Owner의 write server

```
1 lock(Local[p])
2 send(B, page p, copies[p])
3 access[p] = NIL
4 unlock(Local[p])
```

Directory는 먼저 바뀌어도, 실제 WRITE 허용은 무효화 완료 뒤다.

출처: Li & Hudak (1989), Section 4.2, p. 332; Appendix A, pp. 350–351; adapted.

6. 페이지 수신 중 도착하는 후속 요청

1. B가 **local lock을 잡고** 쓰기 요청
2. M이 A로 전달하고 owner 기록을 B로 변경
3. A의 페이지 전송은 지연됨
4. C의 쓰기 요청이 M을 거쳐 B에 도착
5. B의 lock 때문에 C의 요청은 대기
6. B의 fault 처리 완료 후 C의 요청 처리

Directory는 먼저 바뀔 수 있다

`Owner[P] = B`

B가 아직 페이지를 받는 중이어도 가능

$$A \longrightarrow B \longrightarrow C$$

Ownership 이전 순서

메시지 도착 순서가 달라도, local lock이 처리 순서를 지킨다.

출처: Li & Hudak (1989), Section 4.2, p. 332; Appendix A.

6. 완료 확인과 로컬 lock

기본 방식

M이 전달 완료를 기다림.

같은 페이지의 다음 요청은
M의 `Info[p].lock`에서 대기.

개선 방식

M은 새 owner로 바로 전달.

새 owner가 아직 준비 중이면
그 노드의 `Local[p]`에서 대기.

- 읽기: owner가 copy set 등록과 권한 하향을 조율
- 쓰기: 새 owner가 무효화 완료 후 `WRITE` 허용
- 다음 handoff: 새 owner의 fault 처리가 끝날 때까지 대기

후속 요청은 manager의 완료 확인 대신 새 owner의 로컬 lock에서 대기한다.

출처: Li & Hudak (1989), Section 4.2, p. 332; Appendix A, pp. 350–351; correctness intuition.

7. 중앙 manager의 병목

- 공유 가상 메모리: 페이지를 자동으로 이동하고 복제
- 기본 centralized: manager가 위치와 요청 전체를 조율
- Improved centralized: owner에게 copy set과 전달 조율을 위임
- 그러나 page fault의 위치 조회는 여전히 manager를 거친다

분산 관리 방식은 페이지 위치 조회를 여러 노드에 나눈다.

2부: Distributed Manager Algorithms

출처: Li & Hudak (1989), Section 4.2 and opening of Section 5, p. 333.

고정 분산 관리와 브로드캐스트

고정 분산 관리자

$$H(p) = p \bmod N$$

- 페이지별 메타데이터를 담당 관리자에게 배정한다.
- 폴트는 $H(p)$에 요청하고, 관리자는 소유자에게 전달한다.
- 관리 부하는 분산되지만 매핑이 작업 부하에 맞지 않을 수 있다.

브로드캐스트 탐색

- 모든 폴트 요청을 브로드캐스트한다.
- 실제 소유자만 페이지를 제공한다.
- 나머지 프로세서 $N-1$개가 모두 요청을 수신한다.
- 시스템이 커지면 수신 측의 전체 작업량도 늘어난다.

페이지별 담당 관리자는 고정되며, 소유자는 쓰기 요청에 따라 바뀐다.

출처: Li & Hudak (1989), §§5.1–5.2, 333–334쪽.

동적 분산 관리: 소유자 힌트

각 프로세서가 보유하는 페이지별 로컬 상태

$$\mathrm{PTable}[p] = (\mathrm{access}, \mathrm{lock}, \mathrm{copySet}, \mathrm{probOwner})$$

1. 폴트가 발생하면 로컬 `probOwner`가 가리키는 노드에 요청한다.
2. 소유자는 요청을 처리하고, 다른 노드는 자신의 힌트를 따라 전달한다.
3. 전달한 노드는 힌트가 최초 요청 노드를 가리키도록 바꾼다.

힌트가 현재 소유자를 직접 가리키지 않으면 다른 노드를 거쳐 요청을 전달한다.

출처: Li & Hudak (1989), §5.3, 알고리즘 2, 334–336쪽.

소유자 탐색과 힌트 갱신

이전: R의 쓰기 폴트가 세 개의 논리적 요청 간선을 따라간다



완료 후: 방문한 노드들이 새 소유자를 가리킨다



이 그림은 탐색 경로만 보여 준다. 페이지 데이터 응답과 쓰기에 필요한 무효화는 별도의 연산이다.

출처: Li & Hudak (1989), §5.3, 부록 C, 335–336, 352–355쪽; 설명을 위해 재구성한 실행.

소유자 탐색의 정확성

순차적인 폴트 하나가 완료된 뒤

- 정확히 하나의 소유자만 자신을 가리킨다.
- 그 자기 루프를 제거하면 사이클이 없다.
- 모든 프로세서에서 소유자까지의 경로가 유일하다.

여러 폴트가 동시에 발생하면

- 힌트 갱신으로 후속 요청이 먼저 요청한 노드에 도달한다.
- 해당 노드의 페이지 잠금이 후속 요청을 대기시킨다.
- 부록 C는 동시 실행의 정확성을 대응하는 순차 실행으로 설명한다.

소유자까지의 경로가 유지되며, 진행 중인 폴트와 후속 요청은 로컬 lock으로 처리 순서를 맞춘다.

출처: Li & Hudak (1989), 정리 1, 부록 C, 335, 352–357쪽.

소유자 탐색의 분할 상환 복잡도

N: 프로세서 수; K: 한 페이지의 소유자를 찾는 폴트 수.

한 번의 탐색: 최악의 경우

$$L_{\max} \le N - 1$$

전달 경로가 시스템 전체를 지날 수 있다.

연속된 탐색의 전체 비용: 정리 2

$$L_N(K) = O(N + K \log N)$$

$$\frac{L_N(K)}{K} = O\left(\frac{N}{K} + \log N\right)$$

- 참여 프로세서가 고정된 N개라면, 따름정리 1의 로그 항은 N에 의존한다.
- 세는 대상은 **논리적 탐색 요청**이다. 페이지 데이터 응답, 무효화 트래픽, 물리적 홉, 재전송은 제외한다.

이 상한은 읽기 폴트에서도 소유권을 이전하는 알고리즘 2에 적용된다.

출처: Li & Hudak (1989), §5.3, 정리 1–2, 따름정리 1, 부록 C.

동적 분산 알고리즘의 개선

1. **읽기 폴트에서는 소유자를 유지한다.**
   소유권 이전 없이 복사본을 만든다. 논문은 읽기 폴트당 메시지 하나를 줄인다고 설명한다.
2. **현재 소유자를 가끔 브로드캐스트한다.**
   오래된 힌트를 초기화하여 후속 탐색을 줄이지만 브로드캐스트 비용이 든다.
3. **복사본 집합을 트리로 분산한다.**
   다른 읽기 노드에서도 데이터를 받을 수 있고, 무효화는 트리를 따라 전파된다.

정리 2는 알고리즘 2의 상한이다. 읽기 소유권을 유지하는 변형에는 별도의 분석이 필요하다.

출처: Li & Hudak (1989), §§5.3–5.5, 337–341쪽; 부록 C, 354–355쪽.

IVY 실험 환경과 프로그램

실험 환경: Apollo DOMAIN 링, 수정한 Aegis 운영체제, Pascal 프로그램. 사용한 프로세서는 최대 **8개**다.

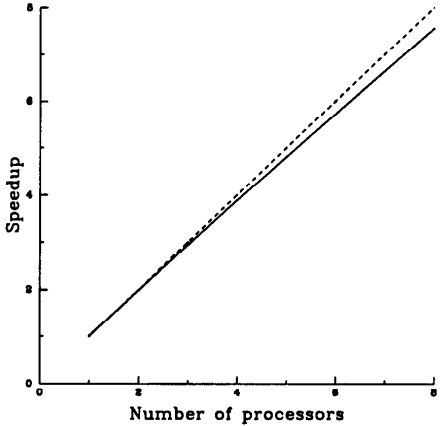
프로그램	문제 크기	주요 특성
3차원 Jacobi PDE	50^3, 40^3	반복 계산; 지역성과 메모리 용량
병합·분할 정렬	레코드 200K개	반복 교환; 이상적 속도 향상도 선형 미만
행렬 곱	128×128	전송한 페이지당 계산량이 많음
벡터 내적	벡터당 원소 128K개	전송한 페이지당 계산량이 적음

$$S(N) = \frac{T(1)}{T(N)} \qquad \text{(프로그램 초기화 시간 제외)}$$

속도 향상비(speedup)를 측정하며, 작업 분할은 프로세서 수에 맞춘다.

출처: Li & Hudak (1989), §§6–6.2, 341–347쪽.

행렬 곱: 페이지당 계산량이 많은 경우



가로축: 프로세서 수 / 세로축: 속도 향상비

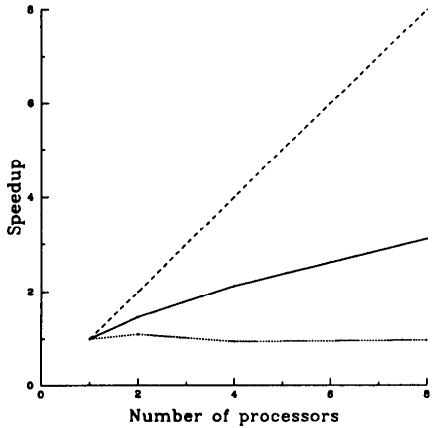
출처: Li & Hudak (1989), 그림 10, §6.2, 347쪽; 원문 그래프.

- $C = AB$, 입력 행렬의 크기는 128×128이다.
- 입력은 한 프로세서에 있다가 필요에 따라 이동한다.
- 측정된 속도 향상은 선형에 가깝다.

데이터를 가져온 뒤 충분히 많은 로컬 계산을 수행하면 공유 데이터 전송 비용을 상쇄할 수 있다.

실선: 측정값. 파선: 이상적인 선형 향상.

벡터 내적: 페이지당 계산량이 적은 경우



가로축: 프로세서 수 / 세로축: 속도 향상비

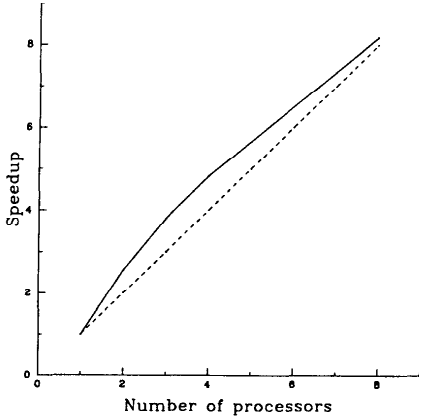
출처: Li & Hudak (1989), 그림 9, §6.2, 346–347쪽; 원문 그래프.

- 각 벡터는 원소 128K개를 가진다.
- 무작위 분산 배치에서는 성능 향상이 제한적이다.
- 두 벡터가 처음에 한 노드에 있으면 성능 향상이 거의 없다.

큰 데이터를 한 번만 읽으면 통신 비용을 상쇄할 계산이 부족하다.

실선: 분산 배치. 점선: 한 노드 배치. 파선: 이상적인 선형 향상.

PDE: 메모리 용량과 초선형 속도 향상



가로축: 프로세서 수 / 세로축: 속도 향상비

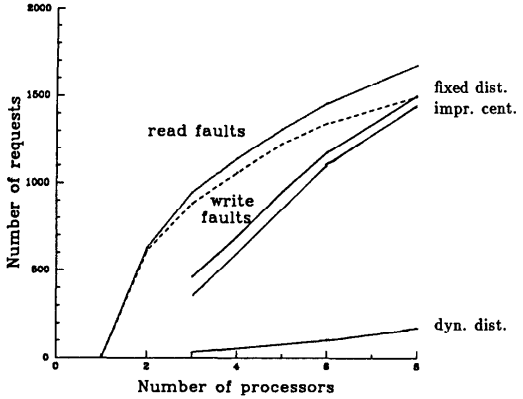
출처: Li & Hudak (1989), 그림 5–7, §6.2, 343–345쪽; 원문 그림 5.

- 50^3 PDE 문제는 한 머신의 물리 메모리에 들어가지 않는다.
- 단일 프로세서 실행에는 많은 디스크 페이징이 발생한다.
- 병렬 실행에서는 합산된 메모리 용량 덕분에 페이징이 줄어든다.

프로세서가 늘면 메모리 용량도 늘어나므로, 단일 노드 실행과 비용 조건이 달라진다.

실선: 측정값. 파선: 이상적인 선형 향상.

PDE: 알고리즘별 요청 전달 횟수



가로축: 프로세서 수 / 세로축: 전달 요청 수
fixed dist.: 고정 분산 / impr. cent.: 개선된 중앙집중형
dyn. dist.: 동적 분산 / read·write faults: 읽기·쓰기 폴트

출처: Li & Hudak (1989), 그림 11, §6.3, 348–349쪽; 원문 그래프.

- PDE의 첫 여섯 반복을 측정했다.
- 알고리즘별 페이지 폴트 수는 비슷하다.
- 동적 힌트는 대체로 소유자를 바로 가리킨다.
- 짧은 시간 동안 한 페이지를 공유하는 프로세서는 적다.

동적 분산 방식에서 소유자를 찾기 위한 전달 요청 수가 줄었다.

실험 결과와 한계

실험으로 확인한 점

- 느슨하게 결합된 네트워크에서도 SVM을 구현할 수 있다.
- 지역성이 좋은 작업 부하는 높은 속도 향상을 얻는다.
- 소유자 힌트가 유효하면 요청 전달 비용이 작다.

결과의 한계

- 최대 8개 프로세서: 대규모 동작은 측정하지 않았다.
- 쓰기 공유와 낮은 데이터 재사용률은 여전히 비싸다.
- 최적의 페이지 크기는 미해결이다.
- 장애 복구는 제시된 알고리즘의 범위 밖이다.

실험에서 탐색 요청 수는 줄었다. 더 큰 시스템에서의 성능은 작업 부하와 네트워크에 따라 달라질 수 있다.

출처: Li & Hudak (1989), §6.3, §7, 348–349쪽; 장애 복구의 한계는 모형에 대한 평가.

결론

1. **공유 가상 메모리는 분산 메모리에 일반적인 읽기·쓰기로 접근하게 한다.**
   MMU 폴트, 이동, 복제, 무효화가 이를 구현한다.
2. **동적 소유자 힌트는 위치 탐색 작업을 분산한다.**
   지역성과 경로 갱신 덕분에 짧은 탐색이 자주 가능하다.
3. **복잡도 분석은 논리적 소유자 탐색 메시지 수를 센다.**
   신뢰성 있는 통신과 최종 응답을 가정하며, 복잡도는 논리적 소유자 탐색 메시지를 센다.

페이지를 가져온 뒤 로컬에서 많이 재사용하는 프로그램에 적합하다.

출처: Li & Hudak (1989), §7, 349쪽; 앞서 설명한 비용 모형과 가정의 범위를 반영.

참고 문헌

Kai Li and Paul Hudak.

Memory Coherence in Shared Virtual Memory Systems.

ACM Transactions on Computer Systems,
제7권 제4호, 321–359쪽, 1989년 11월.

- 원문 파일: `paper.pdf`.
- 슬라이드의 인용 쪽수는 논문에 인쇄된 쪽수를 따른다.
- 실험 그래프는 원문에서 가져왔다.
- 프로토콜 그림과 예시는 설명을 위해 재구성했다.

브로드캐스트의 송신·수신 메시지 수

m: 무효화할 복사본 수; N: 프로세서 수.

무효화 방식	전체 논리 송신 수	전체 논리 수신 수
개별 요청	$2m$	$2m$
브로드캐스트	$m+1$	$N+m-1$
멀티캐스트	$m+1$	$2m$

- 개별 요청: 요청 m개와 응답 m개.
- 브로드캐스트: 한 번의 송신이 다른 프로세서 $N-1$개에 도달하고, 응답 m개를 받는다.
- 멀티캐스트: 한 번의 송신이 복사본 보유자 m개에 도달하고, 응답 m개를 받는다.

논리적인 브로드캐스트 송신 한 번이 여러 수신자의 작업을 유발한다. 이 수만으로 물리 링크의 트래픽을 알 수는 없다.

출처: Li & Hudak (1989), §3.3, 329쪽.

$2K-1$ 상한의 적용 조건

정리 3

브로드캐스트로 모든 프로세서가 현재 소유자를 알게 된 뒤, **서로 다른 프로세서**에서 K번의 페이지 폴트가 발생하면 소유자 탐색에 필요한 논리 메시지 수는

$$1 + 2(K-1) = 2K - 1$$

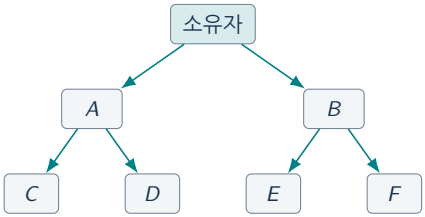
이다.

- 첫 번째 폴트는 소유자를 직접 가리키는 힌트를 따른다.
- 이후의 새 요청 노드는 기존 힌트와 갱신된 전달 경로를 따라간다.
- 이 결과는 가끔 브로드캐스트로 힌트를 갱신하는 방식의 근거가 된다.

같은 프로세서의 반복 폴트에는 이 정리가 적용되지 않는다. 초기화 브로드캐스트와 나머지 폴트 처리 비용도 별도다.

출처: Li & Hudak (1989), §5.4, 정리 3, 337–339쪽; 알고리즘 2의 조건.

복사본 트리와 무효화 비용



복사본 집합의 간선을 따라 무효화를 전파한다

- 각 노드는 자신에게서 복사한 읽기 노드들을 추적한다.
- 유효한 복사본을 가진 노드도 읽기 요청을 처리할 수 있다.
- 쓰기에는 복사본 트리 전체의 무효화가 필요하다.

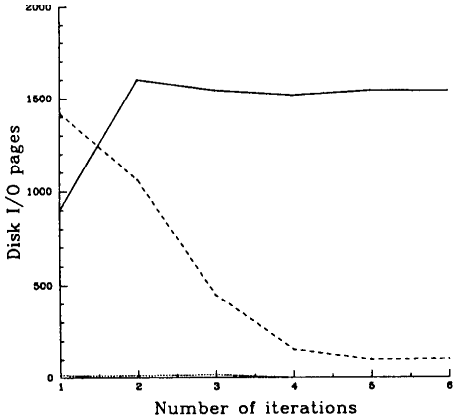
복사본 m개가 균형 트리를 이루면:

$$\text{논리적 깊이} = O(\log m),$$
$$\text{전체 메시지 수} = O(m).$$

무효화를 병렬로 전파하면 대기 시간을 줄일 수 있다. 실제 시간은 트리의 균형과 물리 네트워크 경로에 따라 달라진다.

출처: Li & Hudak (1989), §5.5, 알고리즘 3, 339–341쪽.

PDE: 디스크 페이징 횟수



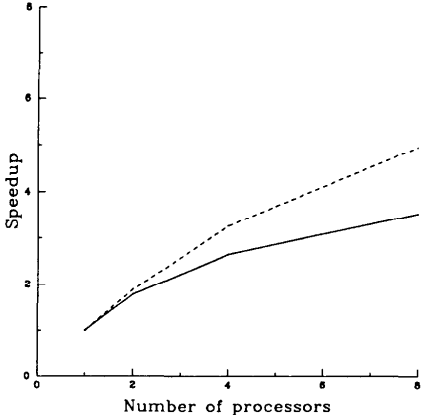
가로축: 반복 횟수 / 세로축: 디스크 I/O 페이지 수

출처: Li & Hudak (1989), 그림 6, §6.2, 344–345쪽; 원문 그래프.

- **실선**: 단일 프로세서 실행에서 페이징이 계속 발생한다.
- **파선**: 2개 프로세서 실행에서 데이터를 초기화한 노드.
- **점선**: 같은 실행의 나머지 노드.

초기 반복 이후에는 합산된 메모리 용량 덕분에 디스크 페이징 부담이 크게 줄어든다.

정렬: 측정값과 이상적인 성능



가로축: 프로세서 수 / 세로축: 속도 향상비

출처: Li & Hudak (1989), 그림 8, §6.2, 346쪽; 원문 그래프.

- 레코드 200K개를 병합·분할 정렬한다.
- 프로세서 수에 따라 작업 분할이 달라진다.
- 이상적인 조건에서도 속도 향상은 선형에 못 미친다.

직선 $S(N) = N$뿐 아니라 알고리즘 자체의 이상적인 성능과도 비교해야 한다.

실선: 측정값. 파선: 모든 메모리 접근 비용이 같을 때의 알고리즘 성능.